

WISH - Digital - Day-2

* Full Name

Vedika Prashant Doe

* WISH Participant ID

WiSH26007

Q1. In the following code, what is the final value of y after simulation?

```
reg [3:0] y;  
initial begin  
y = 4'd2;  
y <= 4'd5;  
y = 4'd7;  
end
```

☐ a) 2

☐ c) 7

☒ b) 5

☐ d) Indeterminate

Q2. Consider the following:

always @(a or b) y = a & b;

If a and b are declared as wire, this code will:

☐ a) Synthesize a 2-input AND gate

☐ c) Infer a latch

☐ b) Produce a compilation error because y must be reg

☒ d) Both a and b depending on y's declaration

Q3. Which correctly describes the difference between always @(*) and always @(a, b, c) when the block reads a, b, and c?

☒ a) They are functionally identical

☐ c) @(*) triggers only on positive edges

☐ b) @(*) includes signals read inside called functions; explicit lists don't

☐ d) Explicit sensitivity lists are faster in simulation

Q4. What does the following produce?

```
always @(posedge clk) begin
```

```
q1 = d;
```

```
q2 = q1;
```

```
end
```

☐ a) A 2-stage shift register

☐ c) A latch on q2

☒ b) Both q1 and q2 become d in the same cycle

☐ d) Undefined behavior

Q5. Intra-assignment delay $y \leq \#5 x$; means:

- ☐ a) Evaluate x after 5 time units, then assign
- ☐ b) **Evaluate x immediately, schedule update to y after 5 time units**
- ☐ c) Delay the always block by 5 time units
- ☐ d) Same as $\#5 y \leq x$;

Q6. Which construct can cause a simulation race condition most easily?

- ☐ a) Non-blocking assignments in a clocked always block
- ☐ b) **Mixing blocking assignments to the same variable from multiple always blocks**
- ☐ c) Using always @(*) for combinational logic
- ☐ d) Using initial for testbench stimulus

Q7. Consider:
always @(en or d)
if (en) q = d;
This infers:

- ☐ a) A flip-flop
- ☐ b) **A latch**
- ☐ c) A mux
- ☐ d) Combinational logic with no memory

Q8. In a fork...join block, statements execute:

☐ a) Sequentially like
begin...end

☒ **b) Concurrently,
and the block
finishes when all
complete**

☐ c) Concurrently, and
the block finishes
when any one
completes

☐ d) Only in
testbenches, not
synthesizable code

Q9. What is wrong with:
function [7:0] add;
input [7:0] a, b;
#2 add = a + b;
endfunction

☒ **a) Functions cannot
have timing
controls**

☐ c) Missing begin...end

☐ d) Nothing — this is
valid

☐ b) Return width
mismatch

Q10. For a synthesizable synchronous reset flip-flop, which is preferred?

☐ a) always @(posedge
clk or posedge rst)

☐ c) always @(*) with
clock in condition

☒ **b) always
@(posedge clk)
with reset checked
inside**

☐ d) Either a or b
produces identical
hardware

Q11. The disable statement is used to:

☐ a) Turn off a module instance

☐ c) Disable a clock signal

☒ **b) Terminate execution of a named block or task**

☐ d) Comment out code at elaboration time

Q12. Which of the following is NOT synthesizable in typical tools?

☐ a) always @(posedge clk) q <= d;

☐ c) case statement with default

☒ **b) wait(enable);**

☐ d) for loop with constant bounds

Q13. Given two always blocks triggered by the same edge, one using = and the other <= on related signals, the outcome is:

☐ a) Deterministic per IEEE 1364

☐ c) Always produces a latch

☒ **b) Potentially a race — mixing styles on shared signals is discouraged**

☐ d) Identical to using only <=

Done

Powered by



Create surveys and forms with ease. [Sign up free.](#)

[Privacy & Cookie Notice](#)